

LeNet-5 CNN Hardware Accelerator on Zynq-7020

The Project

- **A trained neural network turned into a physical circuit** on FPGA fabric
- **Hand-written HLS** — C++ to RTL, not a vendor IP flow
- **Target:** Xilinx Zynq-7020 (XC7Z020, CLG400)
- **Accelerator in the PL**, ARM Cortex-A9 drives it over AXI-Lite

Headline Results

98.97%

FP32 test accuracy

3.08×

latency speedup

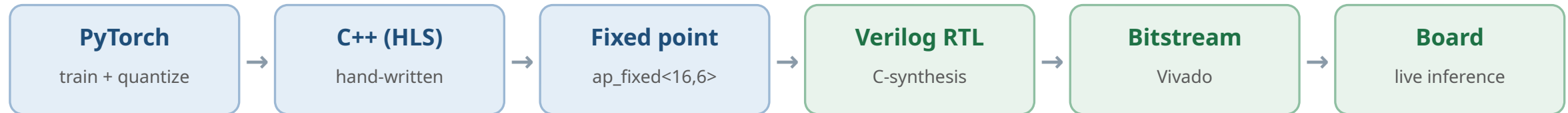
4.65 ms

per inference @ 100 MHz

2.8%

of chip logic used

Design Flow



Every stage was verified against the one before it — the hardware provably computes the same thing as the trained model.

Every Stage — Tool, Input and Output

Stage	Tool used	Input	Output
1 Train	PyTorch (Python)	MNIST — 60,000 training images	Trained weights · 98.97% accuracy
2 Quantize	PyTorch quantization	FP32 trained model	INT8 ONNX · 98.90% accuracy
3 Fit check	Python + Xilinx DS190	Model complexity profile	Verdict: fits on-chip (46.6% BRAM)
4 HLS rewrite	Hand-written C++	Trained weights + architecture	lenet5.cpp — synthesizable C++
5 Fixed point	Vivado HLS ap_fixed<16,6>	FP32 C++ model	16-bit design · 19/20 verified
6 Optimize	Vivado HLS pragmas	1,433,457 cycles	464,699 cycles (3.08× faster)
7 C-synthesis	Vivado HLS 2019.1	lenet5.cpp	Verilog RTL — 25 modules
8 Package IP	Vivado HLS export_design	Verilog RTL	IP core + generated C driver
9 System build	Vivado 2019.1	IP core + Zynq PS + AXI	lenet5_bd_wrapper.bit (3.9 MB)
10 ARM software	Xilinx SDK 2019.1	lenet5_hw.hdf + main.c	executable.elf (182 KB)
11 Deploy	HW Manager + xsct	Bitstream + ELF	Predicted digit printed over UART

Two files leave this pipeline. A 3.9 MB bitstream that configures the FPGA fabric, and a 182 KB executable that runs on the ARM core. Together they run on any machine with a JTAG cable — no project, no source, no rebuild.

Model Training & Hardware Compatibility

LeNet-5 — Trained in PyTorch on MNIST

- **Architecture:** Conv1(1→6) → Pool → Conv2(6→16) → Pool → FC 400-120-84-10
- **10 epochs on MNIST** — 98.97% FP32 test accuracy
- **Chosen over** SqueezeNet, MobileNet and ResNet-18
- Plain sequential convolutions are easiest to hand-pipeline in HLS
- **61,706 parameters** · 418,096 MACs · 291.96 KB memory

Why This Decision Came First

- **If it had not fit**, weights would stream from DDR3 during inference
- **That means tiling, DMA and buffering** — a much harder HLS design
- **Fitting on-chip** let every weight live permanently in BRAM

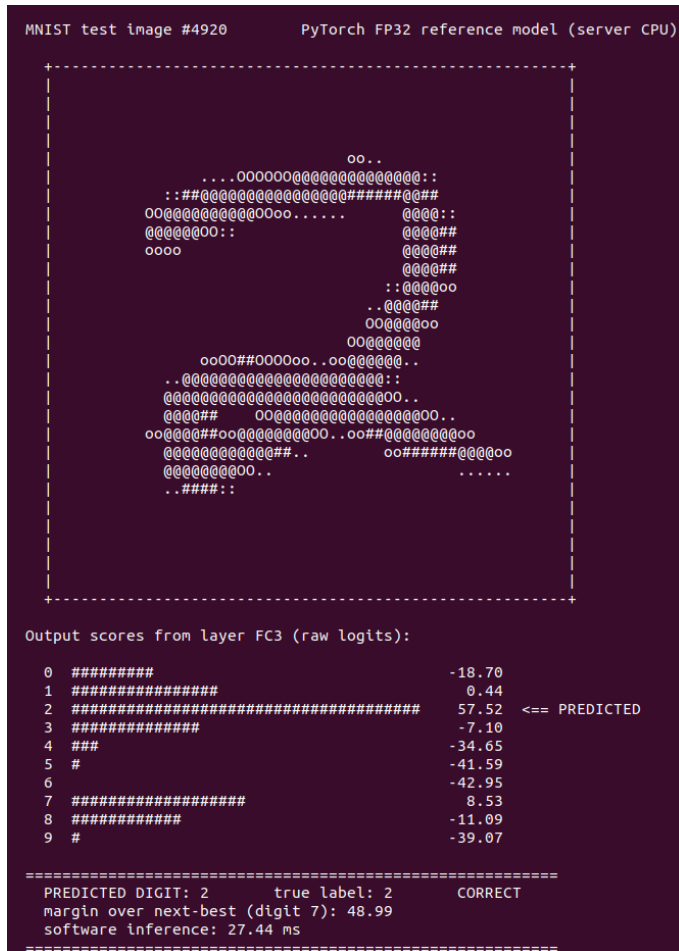
Does It Fit on the Chip?

Resource	Model needs	Chip has	Used
On-chip BRAM	291.96 KB	627.2 KB	46.6%
DSP48E1	418K MACs	220 slices	trivial
LUTs	—	53,200	—
Flip-flops	—	106,400	—

Verdict: Compatible — fits entirely on-chip.

No DDR streaming or tiling needed. This one result simplified the entire hardware design.

Resource figures confirmed against Xilinx DS190. The generic 7-series selection guides do not cover Zynq parts at all.



Input

One 28 × 28 grayscale MNIST test image — 784 pixel values.

Held-out data: these digits were never seen during training. The true label is stored separately and is never passed to the network.

Output

Ten raw scores from the final layer, FC3 — one per digit class.

The largest score is the prediction. The gap to the next-best score shows how confident the network is.

Result Shown

Test image	Predicted	True label	Margin
#4920	2	2 ✓	48.99

The digit is drawn in the terminal from the same 784 values that are fed to the network. Nothing is generated — the image is retrieved from the test set.

Fixed-Point Conversion — FP32 to ap_fixed<16,6>

Why Quantize?

- **Floating point is expensive on FPGA fabric** — more logic, slower
- **ap_fixed<16,6>** — 16 bits total, 6 integer, 10 fractional
- **Values snap to multiples of 1/1024** — nothing exists in between
- **The cost:** a rounding error on every weight, compounding across layers
- **Why it still works:** the prediction is an argmax — only the largest score matters

One Model, Two Implementations

- **The network was trained once**, in PyTorch — there is no second training
- **The C++ was hand-written** and uses those same trained weights
- **Only the arithmetic changed** — both run and agree

Demonstrated live: the same digit through PyTorch and through the compiled fixed-point C++ — the file that was synthesized into the FPGA circuit.

Verification & Resource Impact

Metric	FP32	ap_fixed<16,6>
Predictions matched	reference	19 / 20
BRAM_18K blocks	201 / 280	76 / 280
BRAM utilization	71.8%	27.1%

The one disagreement was not a defect.

On sample 8, PyTorch's own FP32 model predicted the wrong digit. The fixed-point hardware model predicted correctly.

HLS Optimization — Pipelining the Design

What Worked

- **PIPELINE II=1** — a new loop iteration starts every clock cycle
- **Without it**, each iteration finished before the next began
- **Two pragma lines** on the convolution layers gave 2.46×
- **Same treatment on the FC layers** — pipelined end to end
- **No change to the maths** — correctness re-verified at every step

Latency Reduction

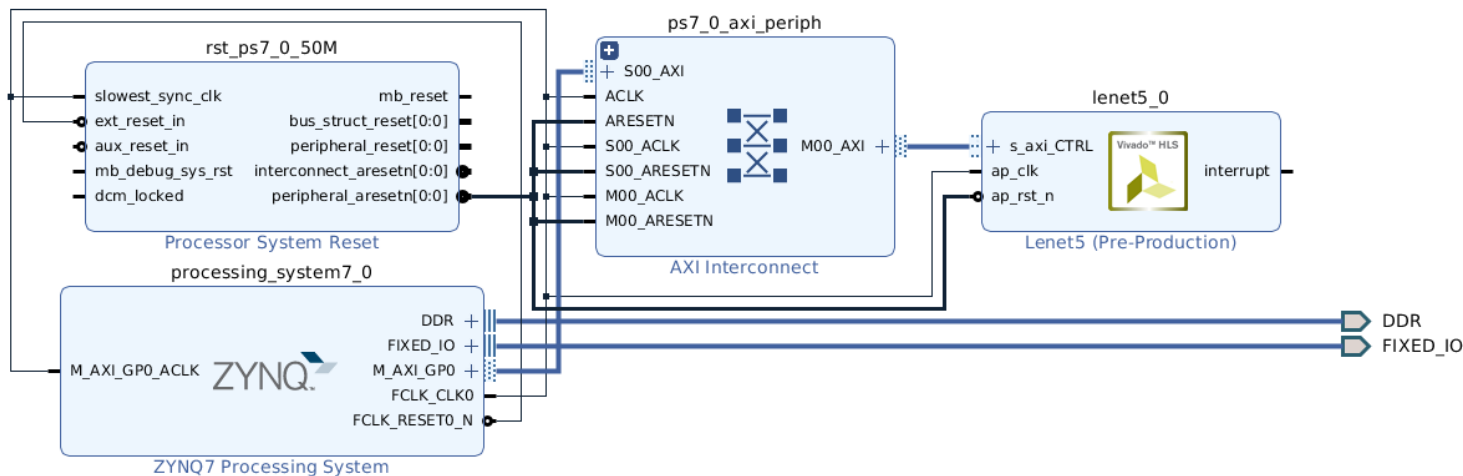
Stage	Cycles	@ 100 MHz
No pragmas (baseline)	1,433,457	14.33 ms
conv1 / conv2 pipelined	582,111	5.82 ms
+ all FC layers pipelined	464,699	4.65 ms

3.08× speedup, DSP usage unchanged at 5 of 220

What Failed — and Why That Matters

- **Full UNROLL**: faster at 366,286 cycles, but violated timing (11.36 ns vs 10 ns) and pushed DSP usage to 81%
- **Root cause**: BRAM has only two read ports — the unrolled loop needed many simultaneous reads
- **ARRAY_PARTITION fixed the ports** but caused a scheduler blowup — one run exceeded five hours with no result
- **Decision**: reverted to the proven pipeline-only design. Reliable beats marginally faster.

System Integration & Implementation Results



Vivado block design — ZYNQ7 Processing System, AXI Interconnect and the custom lenet5 IP at 0x43C0_0000

On the Board

- **Programmed over JTAG** — Vivado Hardware Manager loads the bitstream
- **xsct loads the ARM program**, which prints the predicted digit over UART
- **HLS predicted BRAM and DSP exactly**, but overestimated LUTs by 3.6×

Pipelining over unrolling kept the DSP count low. 97% of both DSPs and LUTs remain unused.

Placed & Routed on Silicon

Resource	Used	Util.
Slice LUTs	1,481	2.78%
Slice Registers	1,483	1.39%
Block RAM	40 / 140	28.6%

5 of 220 DSPs

Timing closed with 11.8 ns slack against a 10 ns target — zero failing endpoints out of 4,535.

The accelerator itself: 1,083 LUTs — 73% of the system logic, and all of its BRAM and DSPs.

LeNet-5 CNN Hardware Accelerator on Zynq-7020

The Project

- **A trained neural network turned into a physical circuit** on FPGA fabric
- **Hand-written HLS** — C++ to RTL, not a vendor IP flow
- **Target:** Xilinx Zynq-7020 (XC7Z020, CLG400)
- **Accelerator in the PL**, ARM Cortex-A9 drives it over AXI-Lite

Headline Results

98.97%

FP32 test accuracy

3.08×

latency speedup

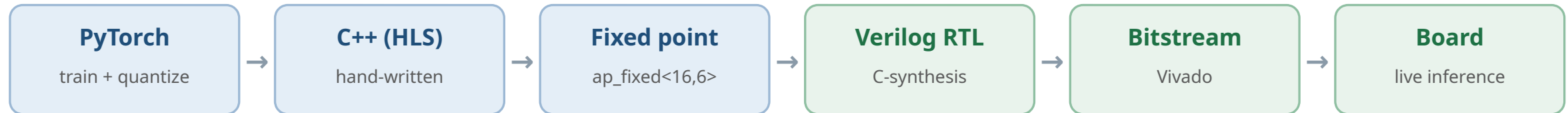
4.65 ms

per inference @ 100 MHz

2.8%

of chip logic used

Design Flow



Every stage was verified against the one before it — the hardware provably computes the same thing as the trained model.

Every Stage — Tool, Input and Output

Stage	Tool used	Input	Output
1 Train	PyTorch (Python)	MNIST — 60,000 training images	Trained weights · 98.97% accuracy
2 Quantize	PyTorch quantization	FP32 trained model	INT8 ONNX · 98.90% accuracy
3 Fit check	Python + Xilinx DS190	Model complexity profile	Verdict: fits on-chip (46.6% BRAM)
4 HLS rewrite	Hand-written C++	Trained weights + architecture	lenet5.cpp — synthesizable C++
5 Fixed point	Vivado HLS ap_fixed<16,6>	FP32 C++ model	16-bit design · 19/20 verified
6 Optimize	Vivado HLS pragmas	1,433,457 cycles	464,699 cycles (3.08× faster)
7 C-synthesis	Vivado HLS 2019.1	lenet5.cpp	Verilog RTL — 25 modules
8 Package IP	Vivado HLS export_design	Verilog RTL	IP core + generated C driver
9 System build	Vivado 2019.1	IP core + Zynq PS + AXI	lenet5_bd_wrapper.bit (3.9 MB)
10 ARM software	Xilinx SDK 2019.1	lenet5_hw.hdf + main.c	executable.elf (182 KB)
11 Deploy	HW Manager + xsct	Bitstream + ELF	Predicted digit printed over UART

Two files leave this pipeline. A 3.9 MB bitstream that configures the FPGA fabric, and a 182 KB executable that runs on the ARM core. Together they run on any machine with a JTAG cable — no project, no source, no rebuild.

Model Training & Hardware Compatibility

LeNet-5 — Trained in PyTorch on MNIST

- **Architecture:** Conv1(1→6) → Pool → Conv2(6→16) → Pool → FC 400-120-84-10
- **10 epochs on MNIST** — 98.97% FP32 test accuracy
- **Chosen over** SqueezeNet, MobileNet and ResNet-18
- Plain sequential convolutions are easiest to hand-pipeline in HLS
- **61,706 parameters** · 418,096 MACs · 291.96 KB memory

Why This Decision Came First

- **If it had not fit**, weights would stream from DDR3 during inference
- **That means tiling, DMA and buffering** — a much harder HLS design
- **Fitting on-chip** let every weight live permanently in BRAM

Does It Fit on the Chip?

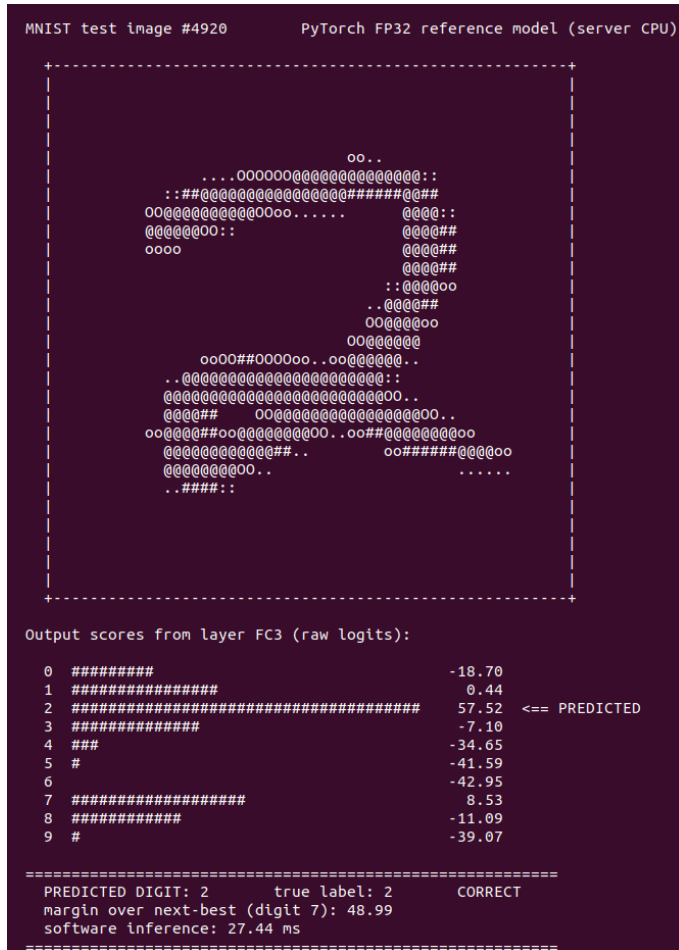
Resource	Model needs	Chip has	Used
On-chip BRAM	291.96 KB	627.2 KB	46.6%
DSP48E1	418K MACs	220 slices	trivial
LUTs	—	53,200	—
Flip-flops	—	106,400	—

Verdict: Compatible — fits entirely on-chip.

No DDR streaming or tiling needed. This one result simplified the entire hardware design.

Resource figures confirmed against Xilinx DS190. The generic 7-series selection guides do not cover Zynq parts at all.

Live Demonstration — Input and Output Defined



Input

One 28 × 28 grayscale MNIST test image — 784 pixel values.

Held-out data: these digits were never seen during training. The true label is stored separately and is never passed to the network.

Output

Ten raw scores from the final layer, FC3 — one per digit class.

The largest score is the prediction. The gap to the next-best score shows how confident the network is.

Result Shown

Test image	Predicted	True label	Margin
#4920	2	2 ✓	48.99

The digit is drawn in the terminal from the same 784 values that are fed to the network. Nothing is generated — the image is retrieved from the test set.

Fixed-Point Conversion — FP32 to ap_fixed<16,6>

Why Quantize?

- **Floating point is expensive on FPGA fabric** — more logic, slower
- **ap_fixed<16,6>** — 16 bits total, 6 integer, 10 fractional
- **Values snap to multiples of 1/1024** — nothing exists in between
- **The cost:** a rounding error on every weight, compounding across layers
- **Why it still works:** the prediction is an argmax — only the largest score matters

One Model, Two Implementations

- **The network was trained once**, in PyTorch — there is no second training
- **The C++ was hand-written** and uses those same trained weights
- **Only the arithmetic changed** — both run and agree

Demonstrated live: the same digit through PyTorch and through the compiled fixed-point C++ — the file that was synthesized into the FPGA circuit.

Verification & Resource Impact

Metric	FP32	ap_fixed<16,6>
Predictions matched	reference	19 / 20
BRAM_18K blocks	201 / 280	76 / 280
BRAM utilization	71.8%	27.1%

The one disagreement was not a defect.

On sample 8, PyTorch's own FP32 model predicted the wrong digit. The fixed-point hardware model predicted correctly.

HLS Optimization — Pipelining the Design

What Worked

- **PIPELINE II=1** — a new loop iteration starts every clock cycle
- **Without it**, each iteration finished before the next began
- **Two pragma lines** on the convolution layers gave 2.46×
- **Same treatment on the FC layers** — pipelined end to end
- **No change to the maths** — correctness re-verified at every step

Latency Reduction

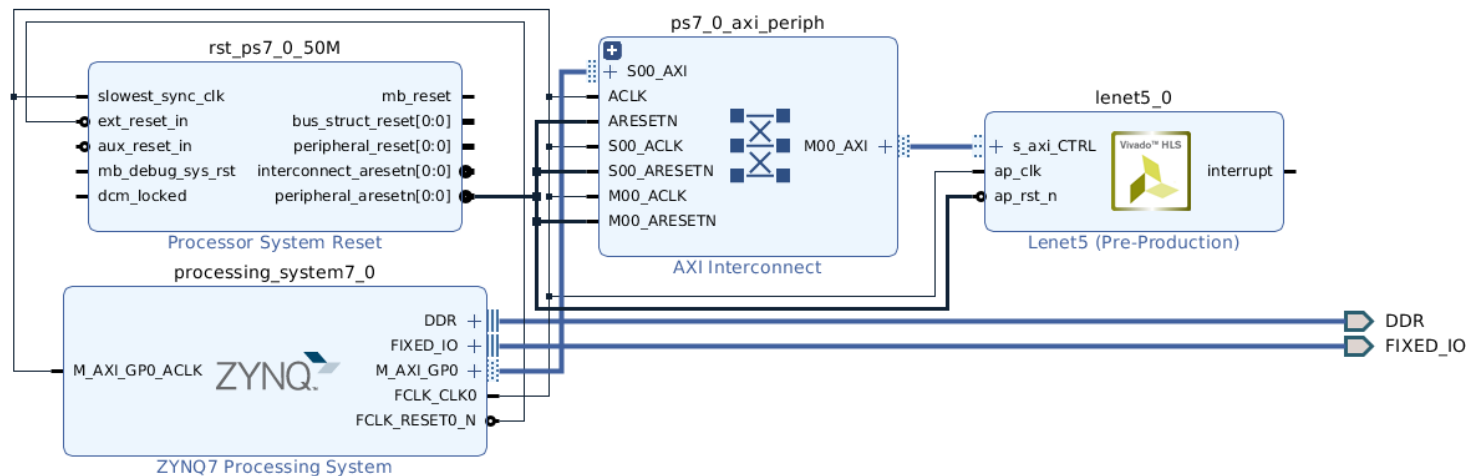
Stage	Cycles	@ 100 MHz
No pragmas (baseline)	1,433,457	14.33 ms
conv1 / conv2 pipelined	582,111	5.82 ms
+ all FC layers pipelined	464,699	4.65 ms

3.08× speedup, DSP usage unchanged at 5 of 220

What Failed — and Why That Matters

- **Full UNROLL**: faster at 366,286 cycles, but violated timing (11.36 ns vs 10 ns) and pushed DSP usage to 81%
- **Root cause**: BRAM has only two read ports — the unrolled loop needed many simultaneous reads
- **ARRAY_PARTITION fixed the ports** but caused a scheduler blowup — one run exceeded five hours with no result
- **Decision**: reverted to the proven pipeline-only design. Reliable beats marginally faster.

System Integration & Implementation Results



Vivado block design — ZYNQ7 Processing System, AXI Interconnect and the custom lenet5 IP at 0x43C0_0000

On the Board

- **Programmed over JTAG** — Vivado Hardware Manager loads the bitstream
- **xsct loads the ARM program**, which prints the predicted digit over UART
- **HLS predicted BRAM and DSP exactly**, but overestimated LUTs by 3.6×

Pipelining over unrolling kept the DSP count low. 97% of both DSPs and LUTs remain unused.

Placed & Routed on Silicon

Resource	Used	Util.
Slice LUTs	1,481	2.78%
Slice Registers	1,483	1.39%
Block RAM	40 / 140	28.6%

5 of 220 DSPs

Timing closed with 11.8 ns slack against a 10 ns target — zero failing endpoints out of 4,535.

The accelerator itself: 1,083 LUTs — 73% of the system logic, and all of its BRAM and DSPs.

Thank You

Questions & Live Demonstration

LeNet-5 on Xilinx Zynq-7020 · Hand-written HLS accelerator